Introduction to Git & GitHub

Date: 04 Feb,2026

What is Version Control?

- VCS is like a smart tool that helps people work together on projects , especially computer programs
- Version Control System manages the entire process of version and keeps the track of the changes made.
- Version Control is a system that:
  - Tracks changes in files overtime
  - Allows reverting to previous versions
  - Helps multiple people work on the same project
  - compare the changes made between any two versions
  - to know information like who, when, why made the changes
- Without version control:
  - Files get overwritten
  - Changes are lost
  - Collaboration becomes difficult

What is Git ?

- GIT is a CLI (Command Line Interface) based tool which is used for taking snapshots of each and every version of our code.
- Git is a distributed version control system used to:
  - Track Source code changes
  - Manage Project history
  - Collaborate with teams
- Key Features:
  - Fast and efficient
  - Works offline
  - Every developer has a full copy of the repository

Why Use Git?

Git helps developers to:

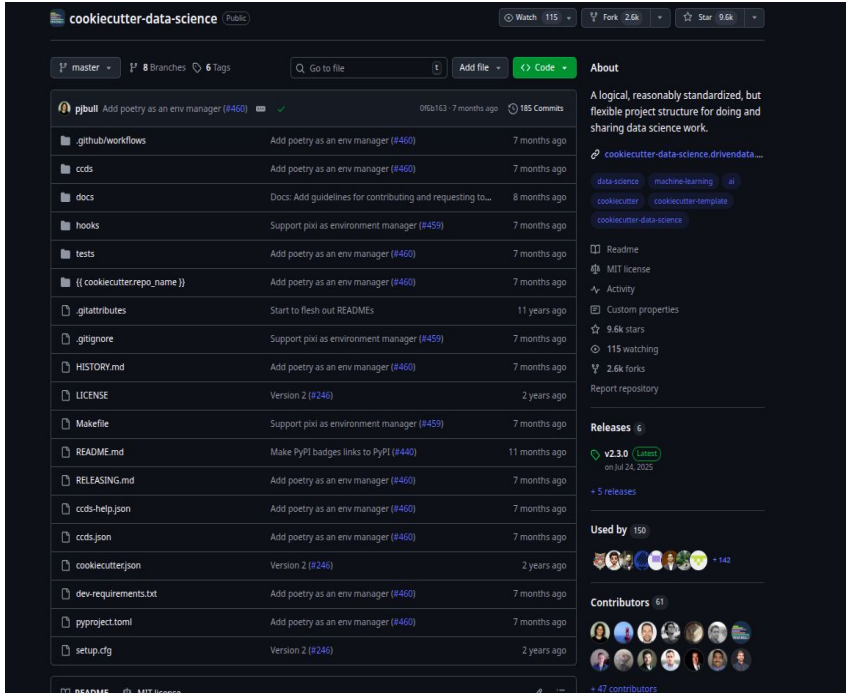
- Track who made what change and when
- Experiment safely using branches
- Fix bugs without breaking main code
- Collaborate efficiently in teams

What is GitHub?

- Github is a web based platform that provides a space where we can collaborate on different projects.
- To host our repository online, we need a server or hosting platform.
- GitHub is a repository hosting platform that uses Git in its core.
- GitHub is a cloud-based platform that:
  - Hosts Git repositories
  - Enables collaboration
  - Provides project management tools
- Features:
  - Repository hosting
  - Collaboration tools
  - Issue Tracking
  - Pull Request
  - Code review
  - Project documentation

Repositories

- Repo = Repositories
- A Folder like structure containing all necessary files
- Can contain folder, images, files etc anything that we want
- Types of repositories:
  - Local repository (on your computer)
  - Remote repository (on GitHub)



Branch

- A branch is a separate line of development.
- If changes in branch are good then merge to main branch. If not make improvements without affecting main codebase
- Why ?
  - Develop features independently
  - Fix bugs safely
  - Avoid breaking main code
- Common branch: **main or master**

Git Workflow (Basic)

- Typical Git workflow:
  - Create or clone a repository
  - Make changes to files
  - Stage changes
  - Commit changes
  - Push changes to GitHub
- This workflow repeats continuously.
- Git states of Files:
  - Modifies → File is changed but not staged
  - Staged → File is ready to be committed
  - Committed → File is saved in Git hiostory

Common Git Commands (Overview)

These are the most commonly used Git commands:

- `git init` → Initialize a new Git repository
- `git clone` → Copy a remote repository to local system
- `git status` → Check the current state of files
- `git add` → Stage changes
- `git commit` → Save changes with a message
- `git push` → Upload local commits to GitHub

- `git pull` → Fetch and merge changes from GitHub

Basic Branch Commands (Conceptual)

- Create a branch → for new feature
- Switch branch → work on that feature
- Merge branch → combine changes into main

What is a Commit Message?

A commit message explains:

- What was changed
- Why it was changed

Good commit messages:

- Are short and clear
- Describe purpose, not code

What is a Pull Request (PR)?

- A **pull request** is like asking your team to look at the changes made in a separate version of the project and consider adding those changes to a main project if theory are good and ready
-
- A Pull Request:
  - Proposes changes
  - Requests review from team
  - Allows discussion before merging
- Used in team collaboration.

Fork

- Fork is making a copy of a repository into our own github account
- We can then do any changes or experiments in that code , that won’t be reflected in the original code.

GitHub in Real Projects

GitHub is used for:

- Open-source projects
- Company codebases
- Portfolio projects
- CI/CD pipelines

Git & GitHub Best Practices

- Commit frequently
- Write meaningful commit messages
- Pull before pushing
- Use branches for features
- Do not push sensitive data